ChatGPT

# Konzept für ein browserbasiertes Abwesenheitsplanungstool (Einzelbenutzer-System)

## Systemübersicht

- **Ziel und Nutzerkreis:** Browserbasiertes Abwesenheitsplanungstool für **einen einzelnen Nutzer (Chefarzt)** zur Verwaltung der Abwesenheiten seines Teams. Mitarbeiter selbst haben **keinen eigenen Zugang** – alle Eingaben erfolgen ausschließlich durch den Chefarzt.
- **Einzelbenutzer-Betrieb: Keine Rollen**, Rechtevergaben oder Stellvertretungen im System. Der Chefarzt hat Vollzugriff auf alle Funktionen; es gibt keine Unterscheidung von Benutzerrollen.
- **Einfaches Login:** Authentifizierung über einen **einzigen Benutzeraccount** (Chefarzt) mittels Benutzername/Passwort. Keine komplexe Benutzerverwaltung notwendig (nur ein Account). Passwort wird sicher (gehasht) in der Datenbank gespeichert.
- **Kernmodule der Anwendung:** Übersichts-Dashboard, Team-Kalender und Berichtssektion bleiben als Hauptbestandteile des UI bestehen. Diese liefern einen schnellen Überblick über Abwesenheiten und Statistiken. (Eine Mitarbeiter-Verwaltungsseite zur Pflege der Personaldaten kommt hinzu, siehe *Funktionen*.)
- **Kein Benachrichtigungssystem: E-Mail-Benachrichtigungen oder interne Notifications entfallen.** Das System versendet keine Mails (z. B. bei neuen Abwesenheiten), da der Chefarzt selbst direkt alle Einträge vornimmt und ein Genehmigungsworkflow im Tool nicht erforderlich ist.
- **Zukunftssicherheit:** Bereits auf Erweiterungen ausgelegt (z.B. geplantes **Rechnungsmodul**), aber aktuelle Version fokussiert ausschließlich auf Abwesenheitsverwaltung. Architekturentscheidungen unterstützen künftige Module, ohne das jetzige System zu verkomplizieren.

## Funktionen (Features)

- **Benutzer-Authentifizierung:** Einfaches **Login** für den Chefarzt. Nach erfolgreicher Anmeldung erhält der Nutzer Zugriff auf alle Bereiche. (Kein Registrierungsvorgang für weitere Benutzer nötig.)
- **Mitarbeiterverwaltung:** Der Chefarzt kann **Personalstammdaten** seines Teams pflegen.
  - **Mitarbeiter anlegen/bearbeiten/löschen:** Erfassung von Mitarbeiterdaten (Name, Position/Abteilung optional, jährliches Urlaubskontingent etc.). Bearbeiten und Entfernen von Mitarbeiterdatensätzen ist möglich.
  - **Übersicht aller Mitarbeiter:** Liste aller angelegten Teammitglieder mit ihren Basisdaten und ggf. verbleibenden Urlaubstagen.
- **Abwesenheitsverwaltung:** Zentrale Funktion zum **Planen und Verwalten von Abwesenheiten** der Teammitglieder.
  - **Abwesenheitseintrag erstellen:** Eingabe einer neuen Abwesenheit für einen Mitarbeiter (Auswahl des Mitarbeiters, Abwesenheitsart, Start- und Enddatum, optional Kommentar). Jede neu erfasste Abwesenheit gilt als **direkt genehmigt** (da vom Chefarzt selbst eingetragen).
  - **Abwesenheit bearbeiten/löschen:** Möglichkeit, bestehende Abwesenheitseinträge zu ändern (z.B. Zeitraum korrigieren) oder zu entfernen. Änderungen werden in der Datenbank versioniert oder mit Zeitstempel aktualisiert (für Nachvollziehbarkeit).
  - **Abwesenheitstypen:** Vordefinierte **Abwesenheitsarten** wie Urlaub, Krankmeldung, Fortbildung etc. stehen zur Auswahl. Diese sind initial fest im System hinterlegt oder als Stammdaten konfigurierbar (für zukünftige Anpassungen durch den Chefarzt, z.B. Hinzufügen neuer Abwesenheitstypen). Farbcodes je Typ sorgen für konsistente Darstellung im Kalender und Berichten.
  - **Konfliktprüfung:** Beim Eintragen prüft das System auf Überschneidungen (gleicher Mitarbeiter bereits abwesend im Zeitraum) und zeigt Warnungen bei Konflikten oder ungültigen Eingaben (z. B. Enddatum vor Startdatum).
- **Team-Kalenderansicht:** Grafische Kalenderübersicht aller Abwesenheiten im Team.

    - **Übersichtliches Kalender-UI:** Darstellung als Gantt-Chart-ähnlicher **Zeitstrahl**: horizontal die Zeitachse (Tage/Wochen), vertikal die Mitarbeiter. Jeder Abwesenheitseintrag erscheint als farbiger Balken beim jeweiligen Mitarbeiter an den entsprechenden Tagen.
    - **Ansichtsoptionen:** Umschaltbar zwischen Monats- und Wochenansicht. Scroll- und Zoom-Funktion für längere Zeiträume. Aktueller Tag wird hervorgehoben.
    - **Detailanzeige:** Bei Hover oder Klick auf einen Abwesenheitsbalken Anzeige von Details (Mitarbeitername, Abwesenheitstyp, Dauer, Kommentar).
    - **Kalenderaktualität:** Änderungen (hinzufügen/löschen von Abwesenheiten) reflektieren sich **sofort** in der Ansicht. Wochenenden und hinterlegte Feiertage werden visuell gekennzeichnet oder ausgeblendet (optional), um Arbeits- vs. Freitage zu verdeutlichen.
- **Dashboard (Übersichtsseite):** Startseite nach dem Login mit **wichtigen Kennzahlen und Hinweisen** für den Chefarzt.
    - **Kacheln/Kennzahlen:** Anzeige von z.B. der **Anzahl der heute abwesenden Mitarbeiter**, dem **nächsten bevorstehenden Urlaubszeitraum**, und ggf. **gesamt genommenen Urlaubstagen im laufenden Jahr**.
    - **Resturlaubsübersicht:** Zusammenfassung pro Mitarbeiter, wie viele Urlaubstage noch verfügbar sind (berechnet aus Urlaubskontingent minus bereits genehmigter Urlaubstage).
    - **Schnellzugriff:** Direkte Links oder Buttons zum **neuen Abwesenheitseintrag** oder zur **Mitarbeiteranlage**, um häufige Aktionen schnell zu erreichen.
- **Berichte und Auswertungen:** Ein eigener Bereich zur **Analyse der Abwesenheitsdaten** über definierte Zeiträume.
    - **Statistikfunktionen:** Erzeugung von Berichten, z.B. **Jahresübersicht pro Mitarbeiter** (Urlaubstage genommen, Krankheitstage), **Abwesenheitsverteilung nach Typ** (z.B. Prozentanteil Urlaub vs. Krankheit im Team), **Monatliche Abwesenheitstage** etc.
    - **Visualisierung:** Grafische Darstellung mittels **Balken- und Kreisdiagrammen** für einen schnellen Überblick. (Beispiel: Balkendiagramm mit Abwesenheitstagen pro Monat, Kreisdiagramm mit Anteil der Abwesenheitstypen.)

    - **Export:** Möglichkeit, einen Bericht als **PDF** herunterzuladen oder auszudrucken, für Dokumentation oder Besprechungen. (Generierung serverseitig als PDF mit Logo/Kopfzeile, optional in Phase 2 umzusetzen.)
    - **Filter:** Interaktive Filtermöglichkeit nach Zeitraum (Jahr, Monat) und Abwesenheitsart, um spezifische Auswertungen zu erhalten.
- **Einstellungen (Basis-Konfiguration):** Einfaches Einstellungsmenü für **Stammdaten und Parameter**.
  *Beispielhafte Inhalte:*
    - **Abwesenheitsarten verwalten:** (Falls nicht fest codiert) Möglichkeit, die Liste der Abwesenheitsarten zu bearbeiten – neue Typen hinzufügen, Farbe zuordnen.
    - **Feiertagskalender hinterlegen:** Eingabe von regionalen Feiertagen, die bei Berechnungen und im Kalender berücksichtigt werden (damit z.B. Wochenenden/Feiertage bei der **Tagezählung** von Abwesenheiten automatisch ausgenommen werden).
    - **Passwort ändern:** Option für den Chefarzt, sein Login-Passwort zu ändern.
    - **(Platzhalter für Rechnungsmodul):** Vorausschauende Integration eines Menüpunktes für das zukünftige Rechnungsmodul (derzeit ohne Funktion), um die Erweiterbarkeit im UI anzudeuten.
- **Nicht implementierte Features:** Funktionen wie **Abwesenheitsantrag durch Mitarbeiter, Genehmigungs-Workflows** oder **automatische Benachrichtigungen** sind im Einzelbenutzer-MVP bewusst **nicht enthalten**. Der Fokus liegt auf direkter Datenerfassung und Übersicht durch den Chefarzt selbst.

## Technische Architektur

- **Client-Server-Struktur:** Trennung von Frontend und Backend. Das **Frontend** ist eine Single-Page Application, die über eine definierte API mit dem **Backend** kommuniziert.
- **Frontend:** Umsetzung mit einer **modernen Frontend-Technologie** – bevorzugt **React** (aktuelle Version) mit **TypeScript** für robuste Typisierung. Styling mit **Tailwind CSS** oder vergleichbarem Utility-CSS-Framework für ein konsistentes, responsives Design. Das Frontend wird als Docker-fähige Build (statische Files) erstellt.

    - **UI-Framework:** React bietet eine komponentenbasierte Architektur, ideal zur Wiederverwendung von UI-Komponenten (Kalender, Formulare, Diagramme etc.). Alternativ wäre auch Vue 3 mit Composition API denkbar – die Wahl richtet sich nach Best Practices und Vertrautheit im Team, aber React + Tailwind gilt als erprobte Kombination für zügige Entwicklung.
    - **State-Management:** Verwendung eines geeigneten State-Managements (z.B. React Context oder Zustand/MobX) für globale Zustände wie angemeldeter Benutzer, Mitarbeiterliste im Speicher, etc. Simpler gehalten, da App-Umfang überschaubar (auf komplexe Stores wie Redux kann evtl. verzichtet werden).
    - **Routing:** Client-seitiges Routing (z.B. mit React Router) für Navigation zwischen Login, Dashboard, Kalender, Berichte, etc. Ungeschützte Routen (Login) vs. geschützte Routen (Hauptanwendung) sind konfiguriert, sodass unautorisierte Zugriffe auf interne Seiten verhindert werden.
- **Backend: Node.js** (LTS-Version) Server mit **Express** Framework (oder alternativ **NestJS** für strukturierteren Ansatz) als REST-API. Implementierung ebenso in **TypeScript** zur besseren Wartbarkeit. Das Backend läuft als eigener Dienst (Docker-Container) und verwaltet Geschäftslogik, Authentifizierung und Datenzugriff.
    - **Schichtenarchitektur:** Klare Trennung der Verantwortlichkeiten im Backend: Routen/Controller für die Entgegennahme von HTTP-Requests, Services/Business-Logik für Verarbeitung (z.B. Berechnung verbleibender Urlaubstage), und Data-Access-Layer mittels ORM für DB-Zugriffe.
    - **ORM & Datenbankzugriff:** Verwendung einer **ORM (Object-Relational Mapping)** Bibliothek zur Abstraktion der MySQL-Datenbank. **Prisma** (empfohlen) bietet eine moderne, typensichere API und passt gut zu TypeScript. Alternativ wäre **Sequelize** möglich – jedoch wird Prisma als zukunftssichere und Docker-freundliche Option bevorzugt (einfache Migrationen, schlankes Schema-Definitionstool). Das ORM verwaltet die Migrationen des Datenbank-Schemas (Erstellen der Tabellen entsprechend dem Datenmodell).
    - **MySQL-Datenbank:** Relationale **MySQL**-Datenbank zur Speicherung aller persistenten Daten (in Docker als separater DB-Container laufend). Verbindungsdetails (Host, User, Passwort) werden über Umgebungsvariablen konfiguriert. Bei Entwicklungsstart ggf. automatische Schemaerstellung/migration via ORM.

    - **REST API:** Das Backend stellt RESTful-API Endpunkte bereit, die vom Frontend per HTTP (JSON) konsumiert werden. JSON wird als Datenformat in Requests/Responses genutzt. Die API-Routen sind thematisch gegliedert (z.B. `/api/auth`, `/api/employees`, `/api/absences`, `/api/reports`).
    - **Session-Handling:** Nach Login erhält der Client einen **JWT-Token** vom Server, der in nachfolgenden Requests zur Authentifizierung mitgesendet wird (im Authorization-Header). Alternativ kann eine serverseitige Session mit Cookie genutzt werden; JWT bietet jedoch Vorteile für eine klare Trennung von Frontend/Backend. Das Backend validiert den Token bei jeder Anfrage und gewährt Zugriff nur mit gültiger Authentifizierung.
- **Deployment & Containerisierung:** Das gesamte System ist auf **Docker**-Betrieb ausgelegt. Docker-Compose kann genutzt werden, um Backend, Frontend (statisches Build) und MySQL-DB als separate Services zu orchestrieren.
    - **Container Setup:** Ein Container für das Node.js Backend, ein Container für die MySQL-Datenbank. Das Frontend wird entweder von einem einfachen Webserver (Nginx/Apache in eigenem Container) ausgeliefert oder via CDN/Hosting (z.B. Vercel/Netlify) bereitgestellt – je nach Infrastrukturvorgabe.
    - **Konfiguration:** Alle sensiblen Einstellungen (DB Credentials, JWT Secret, Port, etc.) sind über **Environment-Variablen** konfiguriert, was Docker-Deployments erleichtert.
    - **Build/Run Automatisierung:** Scripts für den Container-Build und Migrationen (z.B. `docker-compose up` ruft initial DB-Migrationen auf) gehören zum Projekt. Continuous Integration/Deployment-Pipelines können das Container Image bauen und auf den Server deployen.
- **Leistung & Skalierung:** Da es sich um ein Einzelbenutzer-System handelt, sind die Lastanforderungen gering. Die Architektur (getrennte Dienste) ist dennoch gewählt, um **Wartbarkeit und Erweiterbarkeit** zu gewährleisten. Bei Bedarf kann z.B. die Datenbank auf einen Managed Service ausgelagert oder das Backend horizontal skaliert werden, obwohl in diesem Anwendungsfall ein einzelner Container ausreicht.
- **Logging & Monitoring:** Basis-Logging im Backend (z.B. mittels Winston oder eingebauter Logger) für Fehler und wichtige Aktionen (erfolgreiche Logins, DB-Fehler etc.). Bei Deployment in Produktion sollte zudem ein Monitoring (Docker Healthchecks, ggf. einfache Up-Time Überwachung) eingerichtet sein.

# Datenmodell

*(Relationales Modell in MySQL, primäre Tabellen und ihre Felder:)*

- **Benutzer (User):** Enthält den einzigen Benutzeraccount (Chefarzt) zur Authentifizierung.
  - Felder: `user_id` (Primärschlüssel), `username` (Login-Name), `password_hash` (verschlüsseltes Passwort), `name` (optionaler Klarname). *(Hinweis: Bei Einzelbenutzer könnte man diesen Account auch statisch konfigurieren, aber eine Tabelle ermöglicht spätere Erweiterungen, z.B. weitere Benutzer.)*
- **Mitarbeiter (Employee):** Tabelle aller Mitarbeiter/Teammitglieder, deren Abwesenheiten verwaltet werden.
  - Felder: `employee_id` (PK), `name` (Vollname des Mitarbeiters), `position` (Rolle/Abteilung, optional), `vacation_allowance` (jährliches Urlaubskontingent in Tagen, optional für Berechnung Resturlaub), sowie `date_hired` oder `active` Flag falls benötigt.
  - Beziehung: **Keine separate Benutzerkonten** – diese Mitarbeiter haben **keine Login-Daten**, sie dienen rein als Datensätze. (Der Chefarzt selbst könnte bei Bedarf auch als Mitarbeiter erfasst sein, falls seine eigenen Abwesenheiten getrackt würden.)
- **Abwesenheiten (Absence):** Jede Abwesenheit eines Mitarbeiters wird hier als Datensatz gespeichert.
  - Felder: `absence_id` (PK), `employee_id` (FK, Verweis auf **Mitarbeiter**, wessen Abwesenheit), `type` (Art der Abwesenheit, z.B. *Urlaub*, *Krankheit*, *Fortbildung* – entweder als VARCHAR oder als FK auf eine Abwesenheitsarten-Tabelle), `start_date` , `end_date` (Zeitraum der Abwesenheit), `comment` (Textfeld für Notizen, z.B. Vertretungs-Hinweis oder Bemerkung).
  - Zusätzlich möglich: `days_count` (Anzahl betroffener Arbeitstage, kann vom Backend beim Speichern kalkuliert werden, unter Berücksichtigung von Wochenenden/Feiertagen), `created_at` / `updated_at` Timestamp-Felder zur Nachverfolgung.
  - **Status-Feld:** Da das Eintragen durch den Chefarzt die Genehmigung impliziert, ist kein Statuswechsel nötig – **alle Einträge gelten als genehmigt**. Ein `status` -Feld (wie *approved/pending*) kann entfallen oder stets auf *approved* stehen.

- **Abwesenheitsarten (AbsenceType):** *Optional:* Tabelle zur Konfiguration der Abwesenheitstypen.
  - Felder: `type_id` (PK), `name` (z.B. "Urlaub", "Krankheit"), `color_code` (zur UI-Darstellung, Hex-Farbwerte), evtl. `counts_as_vacation` (Bool, ob von Urlaubskontingent abzuziehen).
  - Diese Tabelle ist nicht zwingend – Typen könnten auch als feste Enums im Code definiert sein. Die Tabelle bietet jedoch Flexibilität, ohne Codeänderung neue Typen hinzuzufügen (z.B. "Home-Office Tag"). Im MVP können Standardtypen fest vorgegeben werden.
- **Feiertage (Holidays):** *Optional:* Tabelle zur Hinterlegung von Feiertagen/Ferientagen, um automatische Ausblendung in Berechnungen zu ermöglichen.
  - Felder: `date` (Datum des Feiertags), `name` (Name, z.B. "Weihnachten"). Diese Daten könnten auch extern gehalten oder per Konfiguration geladen werden. Für Berechnungen (z.B. Abwesenheitsdauer) und Kalenderdarstellung werden diese Daten herangezogen.
- **(Geplant) Rechnungen (Invoices):** *Platzhalter für zukünftiges Modul.* Sollte später ein Rechnungsmodul integriert werden, kämen neue Tabellen hinzu, z.B.:
  - `invoices` : mit Feldern `invoice_id` , `date` , `amount` , `recipient` etc., und ggf. Verknüpfung zu Mitarbeitern oder Projekten. Aktuell noch nicht umgesetzt, aber das Datenmodell ist vorbereitet, um neue Entitäten mit Beziehungen (z.B. ein Mitarbeiter als Rechnungsempfänger) ergänzen zu können.

*(Alle Primärschlüssel als Auto-Increment. Fremdschlüssel zwischen `Absence.employee_id` → `Employee.employee_id` gewährleisten referenzielle Integrität. Indexe auf häufig gefilterten Feldern wie Datum sind vorgesehen. Verwendung von ORM-Migrationen stellt konsistente Schema-Updates sicher.)*

## UI-Komponenten

- **Hauptnavigation:** Konsistente Navigationsleiste (Sidebar oder Topbar) mit den Hauptbereichen: **Dashboard**, **Kalender**, **Mitarbeiter**, **Abwesenheiten** (wenn nicht mit Kalender kombiniert) und **Berichte**. Zusätzlich ein Menü für **Einstellungen** und **Logout**.

    - Bei Verwendung einer Sidebar wird der aktuelle Bereich hervorgehoben. Da nur der Chefarzt Zugang hat, sind **alle Menüpunkte stets sichtbar** (kein rollenspezifisches Ausblenden erforderlich).
- **Login-Seite:** Simples Login-Formular (Benutzername, Passwort, Login-Button). Nach Login wird ein Token gespeichert (z.B. im localStorage oder als Cookie) und der Nutzer auf das Dashboard weitergeleitet. Gestaltung schlicht, mit Klinik-Logo oder Produktlogo.
- **Dashboard-Komponenten:** Übersichts-Kacheln (Cards) für Kennzahlen, z.B. als **Info-Boxen** mit Icons. Eine **Liste kommender Abwesenheiten** (z.B. "Wer ist in den nächsten 7 Tagen abwesend?") als kompakte Tabelle. Mögliche **Aktionselemente**: Button "Neue Abwesenheit erfassen", der direkt das Eingabeformular öffnet.
- **Kalender-Komponente:** Zentrales UI-Element für die Team-Kalenderansicht. Implementiert als **scrollbare Tabelle oder Grid**: linke Spalte listet Mitarbeiter, Spalten pro Tag/Woche. Abwesenheiten werden als farbige Balken über die entsprechenden Zellen dargestellt.
    - Unterstützt **Interaktion**: Tooltip/Popover bei Klick für Details, ggf. Drag-and-Drop-Funktionalität zum Verschieben/Verlängern einer Abwesenheit (optional).
    - Legende oder Filter für Abwesenheitsarten (Farbcodes) außerhalb des Kalenders.
    - Bei schmalem Bildschirm (mobil) erfolgt Umschalten auf eine vereinfachte Listen- oder Monatskalenderansicht pro Mitarbeiter (Responsive Design Vorsorge, auch wenn primär Desktop-Ziel).
- **Mitarbeiterverwaltung UI:** Formular zur **Erfassung eines Mitarbeiters** (Eingabefelder: Name, evtl. Auswahl Abteilung, Urlaubskontingent, etc.) und **Tabelle aller Mitarbeiter**. In der Tabelle je Zeile ein Mitarbeiter mit Aktionen **Bearbeiten** (öffnet Formular) und **Löschen**. Validierung bei Eingabe (z.B. Pflichtfeld Name).
- **Abwesenheits-Formular:** Dialog oder eigenständige Seite zur **Erfassung/Bearbeitung einer Abwesenheit**. Enthält Eingabekomponenten:
    - Dropdown für **Mitarbeiterauswahl** (Liste der Namen),
    - Dropdown oder Radio für **Abwesenheitstyp**,
    - **Datumsfelder** Start/Ende (idealerweise mit Date-Picker-Kalender zur einfachen Datumsauswahl),

    - optionales Textfeld **Kommentar**.
    - Buttons: **Speichern** und **Abbrechen**.
    - Bei Bearbeitung eines bestehenden Eintrags sind die Felder vorbefüllt. Bei neuem Eintrag können Felder wie Enddatum automatisch gesetzt werden (z.B. gleich Startdatum bei eintägiger Abwesenheit, mit Möglichkeit zur Anpassung).
- **Abwesenheitsliste:** Zusätzlich zum Kalender eine tabellarische Übersicht aller Abwesenheiten (bzw. filterbar nach Mitarbeiter oder Jahr), um Daten auch in Listenform zu sehen. Spalten etwa: Mitarbeitername, Typ, Zeitraum (Start–Ende), Dauer in Tagen, Kommentar. Nützlich für schnelle Suche/Sortierung (z.B. alle Krankmeldungen im Monat). Diese Liste ist insbesondere für exportierbare Berichte oder Admin-Zwecke hilfreich.
- **Berichtskomponenten:** Interaktive **Diagramm-Widgets** (z.B. Balkendiagramm, Tortendiagramm) zur Visualisierung der Abwesenheitsstatistik. Eingebettet z.B. mit Chart-Bibliothek (Chart.js, D3.js o.ä.). Außerdem **Filter-Steuerelemente** (Dropdowns oder Date-Picker) um den anzuzeigenden Zeitraum oder bestimmte Mitarbeiter einzugrenzen. Ergebnisse als **dynamische Grafiken und tabellarische Werte** angezeigt.
    - Bei komplexeren Auswertungen können auch mehrere Diagramme nebeneinander dargestellt werden (z.B. links Jahresübersicht pro Monat, rechts Verteilung nach Typen).
- **Responsive Design:** Die Web-Anwendung wird primär für Desktop entworfen (große Tabellen/Charts). Dennoch werden **responsive Breakpoints** berücksichtigt: Navigation als ausklappbares Menü auf mobilen Geräten, Kalender wechselt ggf. auf kompaktere Ansicht. Dies wird mit flexiblen CSS-Grid/Flexbox-Layouts und Tailwind-Klassen umgesetzt. (Volle mobile Optimierung kann nachrangig behandelt werden, Fokus MVP auf Desktop.)
- **Konsistentes Design:** Verwendung eines einheitlichen Farbschemas (z.B. Klinik-Blau als Primärfarbe, Signalfarbe für Abwesenheitsarten) und klarer **Icons/Symbole** für Aktionen (z.B. "+" für hinzufügen, Stift für bearbeiten, Papierkorb für löschen). Alle UI-Komponenten sind modular, wiederverwendbar und folgen einer einfachen, intuitiven Benutzerführung (z.B. Formulare mit klar beschrifteten Feldern und Buttons, Kalender mit eindeutiger Legende).

## APIs (Schnittstellenentwurf)

- **Authentifizierungs-API:**
  - `POST /api/auth/login` – Prüft Anmeldedaten (Benutzername, Passwort). Bei Erfolg Rückgabe eines JWT-Tokens (oder Setzen eines Auth-Cookies) zur weiteren Nutzung der API.
  - `POST /api/auth/logout` – (optional, falls JWT: im Frontend durch Token-Löschung erledigt; falls Session: Server invalidiert die Session).
- **Mitarbeiter-API:** *(Schutz: nur mit gültigem Login-Token zugänglich)*
  - `GET /api/employees` – Liste aller Mitarbeiter abrufen (für Mitarbeiterübersicht und Dropdowns).
  - `POST /api/employees` – Neuen Mitarbeiter anlegen (erwartet JSON mit Name, evtl. Position, Urlaubskontingent...).
  - `PUT /api/employees/{id}` – Vorhandenen Mitarbeiter bearbeiten (Änderung von Name, Kontingent etc.).
  - `DELETE /api/employees/{id}` – Mitarbeiter entfernen (nur möglich, wenn keine Abwesenheiten mehr verknüpft, oder es werden auch die Abwesenheiten gelöscht/archiviert – je nach Geschäftsregel).
    *(Hinweis: Da es nur einen Nutzer gibt, erfolgt keine Rollenprüfung – der angemeldete Chefarzt darf alle diese Aktionen durchführen.)*
- **Abwesenheits-API:**
  - `GET /api/absences` – Abfragen aller Abwesenheiten. Optional mit Query-Parametern filterbar (z.B. `?year=2025` oder `?employeeId=X`). Standard gibt z.B. aktuelle und zukünftige Abwesenheiten zurück.
  - `POST /api/absences` – Neuen Abwesenheitseintrag erstellen. Erwartet JSON mit Mitarbeiter, Typ, Start-/Enddatum, Kommentar. Server prüft auf Konflikte und Geschäftslogik (z.B. gültiger Mitarbeiter, Enddatum >= Startdatum) und speichert bei Erfolg den Datensatz.
  - `PUT /api/absences/{id}` – Abwesenheitseintrag aktualisieren (z.B. Datumsänderung oder Kommentar ergänzen).
  - `DELETE /api/absences/{id}` – Abwesenheit löschen (Entfernen aus dem Kalender).

    - **Kalenderdatengenerierung:** Evtl. ein spezieller Endpunkt `GET /api/absences/calendar?from=YYYY-MM-DD&to=YYYY-MM-DD`, der aggregierte Abwesenheitsdaten für den Kalender liefert (z.B. gruppiert nach Mitarbeiter mit deren Abwesenheiten im Zeitraum). Dies kann jedoch auch vom Frontend über `/absences` selbst zusammengesetzt werden.
- **Berichte/Statistik-API:**
    - `GET /api/reports/summary?year=YYYY` – Gesamtsummen und Statistiken für ein Jahr abrufen (z.B. Urlaubstage pro Mitarbeiter, Krankheitstage gesamt etc.).
    - `GET /api/reports/employee/{id}?year=YYYY` – Detaillierter Bericht für einen Mitarbeiter/Jahr (Liste aller Abwesenheiten und Summe der Tage nach Typ).
    - *Optional:* `GET /api/reports/export?type=YEARLY_SUMMARY&year=YYYY` – Generiert einen PDF-Bericht und liefert ihn als Download (Content-Type application/pdf). Alternativ könnte der PDF-Export auch vom Frontend mittels einer Bibliothek erfolgen, daher optional serverseitig.
- **Stammdaten-API:**
    - `GET /api/absence-types` – (falls AbsenceType Tabelle vorhanden) Liste der Abwesenheitsarten abrufen inkl. Farbcode.
    - `POST/PUT/DELETE /api/absence-types...` – Änderungs-Endpoints für Abwesenheitsarten (nur falls Chefarzt diese in UI bearbeiten kann; ansonsten entfallen diese API-Routen im MVP).
    - `GET /api/holidays?year=YYYY` – Liste der hinterlegten Feiertage abrufen (für Frontend, um im Kalender anzuzeigen). Anlage der Feiertage kann entweder per Script erfolgen oder via separatem Admin-Endpoint/DB-Seeding (im MVP evtl. Hardcoded in Backend).
- **Architektur der API:** Die API folgt REST-Konventionen und gibt HTTP-Statuscodes und JSON zurück. Z.B. 200 für erfolgreiche GET, 201 Created bei erfolgreichem POST (mit zurückgelieferter neuer Ressource oder ID), 400 bei Validierungsfehler (inkl. Fehlermeldung), 401 bei fehlender Authentifizierung. Die Requests werden serverseitig validiert (kein Verlass nur auf Client-Validierung).

    - **Beispiel Workflow:** Frontend ruft beim Laden des Kalenders `GET /api/employees` (für Mitarbeiterliste) und `GET /api/absences?year=2025` ab. Bei Klick "neue Abwesenheit" sendet Frontend `POST /api/absences` mit den Eingabedaten; Backend speichert und gibt 201 + JSON des neuen Eintrags, Frontend aktualisiert daraufhin Kalender und andere betroffene Daten.
- **Keine externe Integration im MVP:** Das System ist in sich geschlossen – es gibt vorerst keine Schnittstellen zu externen Systemen (wie HR-Systeme oder Kalender-Sync). Alle Daten werden intern verwaltet. Dies vereinfacht die API und reduziert Abhängigkeiten in der ersten Version.

## Sicherheit

- **Authentifizierungsschutz:** Alle internen API-Routen (Mitarbeiter, Abwesenheiten, Berichte) sind **nur nach Login zugänglich**. Nicht angemeldete Aufrufe erhalten HTTP 401. Im Frontend wird bei fehlendem Login-Token automatisch zum Login redirectet.
- **Passwort-Sicherheit:** Das Chefarzt-Passwort wird **verschlüsselt (gehasht + gesalzen)** in der Datenbank gespeichert (z.B. mittels bcrypt). Es gibt keine Passwortrücksetz-Funktion im MVP (da einziger Nutzer, Passwortänderung erfolgt eingeloggt in den Einstellungen). Default-Zugang wird initial gesetzt und sollte vom Chefarzt geändert werden können.
- **JWT/Session Security:** Bei JWT-Verwendung wird ein ausreichend komplexes **Secret** verwendet und das Token hat eine sinnvolle Ablaufzeit (z.B. 8 Stunden). Token werden nur über HTTPS übertragen. Alternativ bei Session-Cookie: Secure, HttpOnly Cookies einsetzen, um XSS und CSRF-Risiken zu minimieren.
- **Eingabevalidierung:** Alle vom Benutzer eingegebenen Daten (Login-Daten, Formulareingaben für Mitarbeiter/Abwesenheiten) werden **serverseitig validiert**. Dies verhindert SQL-Injection (wobei der Einsatz eines ORM ohnehin vorbereitete Queries nutzt) und stellt sicher, dass z.B. Datumsfelder korrekt sind. Ungültige Eingaben führen zu klaren Fehlermeldungen.
- **Zugriffsschutz & Berechtigung:** Da nur ein Nutzer existiert, ist komplexes Role-Based-Access-Control hinfällig. Dennoch prüft das System die **Zugriffsrechte** pro Request (z.B. via Middleware, die den JWT prüft). Es wird verhindert, dass API-Aufrufe ohne gültige Authentifizierung oder mit manipulierten Tokens durchgeführt werden können.

- **Verschlüsselung & Verbindungen:** Der Dienst soll über HTTPS bereitgestellt werden (gerade bei externem Hosting), um Login-Daten abhörsicher zu übertragen. In Docker-Umgebung kann ein Reverse Proxy (z.B. Traefik, Nginx) SSL terminieren. Innerhalb des internen Netzwerks (zwischen Frontend, Backend, DB) genügt die interne Kommunikation, aber Best Practice ist auch hier Absicherung der DB-Zugriffe (Firewall, kein offener DB-Port nach außen).
- **Error Handling:** Sensible Fehler (z.B. DB-Errors) werden serverseitig geloggt, aber dem Client wird eine generische Fehlermeldung gegeben, um keine Interna preiszugeben. Rate-Limiting ist hier nicht kritisch (nur ein Nutzer), aber grundlegender Schutz vor Brute-Force-Login könnte implementiert werden (z.B. nach X Fehlversuchen kurze Sperre).
- **Datenintegrität & Backup:** Regelmäßige Backups der MySQL-Datenbank (z.B. jede Nacht) sind vorgesehen, um bei Datenverlust die Abwesenheits- und Personaldaten wiederherstellen zu können. Integrität wird zudem durch referenzielle Constraints gewährleistet (z.B. kein Abwesenheitsdatensatz ohne gültigen Mitarbeiter).
- **Keine externen Nutzer – Datenschutz:** Das System speichert Mitarbeiterdaten (Name, Abwesenheiten). Diese Daten sind nur für den Chefarzt einsehbar. Es gelten interne Datenschutzrichtlinien; da keine externen Zugriffe oder Cloud-Speicher genutzt werden, verbleiben die Daten in der Kontrolle der Klinik. (Optional könnte eine einfache Datenschutzhinweis-Seite eingebunden werden, obwohl das Tool primär intern genutzt wird.)

## Erweiterbarkeit

- **Geplantes Rechnungsmodul:** Die Architektur berücksichtigt von Anfang an die **Integration eines Rechnungsmoduls** in zukünftigen Versionen. Z.B. kann in der Navigation ein weiterer Punkt **"Rechnungen"** ergänzt werden, und im Backend neue Routen ( `/api/invoices` ) und Datenbanktabellen (Rechnungen, Kunden etc.) können ohne Bruch hinzugefügt werden. Durch die modulare Frontend-Struktur (getrennte Seiten/Komponenten) lässt sich ein neues UI-Modul für Rechnungen implementieren, das parallel zum bestehenden Abwesenheitsteil läuft. Datenbankseitig sind Relationen zu bestehenden Tabellen (z.B. Mitarbeiter als Rechnungsempfänger) möglich, da das Schema relational aufgebaut ist.

- **Modulare Code-Struktur:** Das Projekt ist in **funktionale Module** gegliedert (z.B. Services für Mitarbeiter, Abwesenheiten, später Rechnungen). Neue Funktionen können als neue Module mit minimalen Seiteneffekten ergänzt werden. Beispiel: die **Trennung von Frontend-Routen und Backend-Controllern** pro Feature erleichtert es, neue Features hinzuzufügen. In Express/NestJS können neue Controller/Router für das Rechnungsmodul erstellt werden, ohne bestehende zu ändern. Im Frontend kann ein neues React-Routing für Rechnungen ergänzt werden.
- **Skalierung auf Mehrbenutzer:** Sollte künftig doch ein **Multi-User-Szenario** gewünscht sein (z.B. Mitarbeiter erhalten Lesezugriff oder Stellvertreter-Funktion wird eingeführt), ist das System darauf vorbereitbar. Die Datenbank besitzt bereits eine User-Tabelle. Die Rollenlogik müsste erweitert werden (z.B. ein `role`-Feld und entsprechende Berechtigungsprüfung in der Middleware). Das UI- und API-Design kann um einfache **Rollenprüfungen** ergänzt werden. Durch die initiale Einfachheit ist ein späteres Hinzufügen eines Rollenmodells zwar mit Aufwand verbunden, aber die saubere Trennung (z.B. Auth-Middleware, Navigationsberechtigungen) erleichtert die Umsetzung.
- **Benachrichtigungssystem nachrüstbar:** Falls in Zukunft E-Mail-Benachrichtigungen oder z.B. **ICS-Kalender-Exporte** gewünscht sind, kann dies als eigenständiger Service oder Modul hinzugefügt werden. Das Backend ließe sich um einen Mail-Service erweitern (z.B. Integration von Nodemailer), der an bestimmten Hooks (z.B. neue Abwesenheit erfasst) Mails verschickt. Da momentan darauf verzichtet wird, entsteht hier keine enge Verkettung – ein späteres Hinzufügen beeinflusst nicht die Kernfunktionalität.
- **Technologie-Updates:** Die gewählte Tech-Stack (Node, React, MySQL) ist etabliert und langfristig unterstützt. Updates (z.B. Node LTS-Wechsel, Security-Patches) können durch Docker-Image Updates und Neu-Deployment einfach eingespielt werden. Durch Verwendung von ORM und hohen Abstraktionsgrad im Code ist auch ein Wechsel der Datenbank (z.B. zu PostgreSQL) mit moderatem Aufwand möglich, sollte dies nötig sein.

- **Wartbarkeit & Weiterentwicklung:** Klare **Dokumentation und sauberes Coding** (z.B. via README, kommentierter Code, Definition von Coding-Standards) stellen sicher, dass weitere Entwickler oder KI-Codex-Assistenten schnell neue Tickets umsetzen können. Jeder der oben definierten Abschnitte (Funktionen, UI, API) kann in einzelne Entwicklungs-Tickets heruntergebrochen werden. Damit ist eine agile Weiterentwicklung (z.B. Hinzufügen eines neuen Berichtstyps oder einer neuen Eingabefeldes) jederzeit möglich, ohne das Gesamtsystem zu destabilisieren.
- **Resümee:** Dieses Konzept legt ein robustes Fundament für ein schlankes Abwesenheitsplanungs-Tool. Es erfüllt alle aktuellen Anforderungen des Chefarztes und lässt gleichzeitig genug Spielraum, um zukünftige Bedürfnisse – vom Rechnungsmodul bis hin zu erweiterten Benutzerrollen – nahtlos integrieren zu können. Alle Entscheidungen (Tech-Stack, Architektur, Datenmodell) sind darauf ausgerichtet, eine **klare, erweiterbare und wartungsfreundliche** Anwendung zu ermöglichen, aus der unmittelbar ein Pflichtenheft und konkrete Implementierungsaufgaben abgeleitet werden können.

---

### Quellenangaben

**Konzept für ein browserbasiertes Abwesenheitsplanungstool (Chefarzt + Tea...**
file://file-NFgzEYGuEkQToxgKz6twnj

**Konzept für ein browserbasiertes Abwesenheitsplanungstool (Chefarzt + Tea...**
file://file-NFgzEYGuEkQToxgKz6twnj

**Konzept für ein browserbasiertes Abwesenheitsplanungstool (Chefarzt + Tea...**
file://file-NFgzEYGuEkQToxgKz6twnj

### Alle Quellen

Konzept ...Team).pdf